

KRR Project — Milestone 3

From Mapping to SHACL and Demonstrator

Validation, Querying, and Interactive Demonstration

ANKRI Mohamed-Khalil · MICHON Charlotte · PAULIS Antoine

Academic Year 2025–2026

PART 1 — SHACL Validation

Once the R2RML mapping has been executed and the out.ttl knowledge graph has been generated, the next step is to **validate** it. Generating RDF triples does not guarantee correctness — the data could be missing mandatory properties, point to the wrong class, or contain malformed values. SHACL provides a formal mechanism to verify these structural requirements.

1. What is SHACL?

SHACL (Shapes Constraint Language) is a W3C standard for validating RDF graphs against a set of structural rules called **shapes**. A shape describes what triples an individual of a given class must, may, or must not have.

SHACL operates on two graphs simultaneously:

- **The data graph** — the RDF knowledge graph to be validated (out.ttl in this project).
- **The shapes graph** — the file containing the constraint definitions (shapes.ttl in this project).

The validator checks every individual in the data graph against the shapes that target its class, and produces a **validation report**. If all constraints are satisfied, the report contains sh:conforms true. If any constraint is violated, the report lists each violation with the affected node, the violated constraint, and the custom error message.

1.1 SHACL vs SPARQL — Complementary Roles

	SHACL	SPARQL
Purpose	Validate structure and constraints	Query and retrieve data
Question asked	Is this data well-formed?	What does this data contain?
Output	Validation report (pass/fail + violations)	Result set (rows and columns)
When used	After mapping, before querying	After validation
Analogy	Unit tests for your data	SELECT queries on a database

2. Anatomy of a SHACL NodeShape

A NodeShape targets all individuals of a given class and defines a set of property constraints. Each constraint is expressed as an `sh:property` block with parameters describing what is required.

Here is the complete shape for the Title class, annotated:

```
shape:TitleShape a sh:NodeShape ;
  sh:targetClass ont:Title ;           # applies to all ont:Title individuals
  sh:name "TitleShape" ;
  sh:description "Every Title must have a primaryTitle and a ContentType." ;

  sh:property [
    sh:path ont:primaryTitle ;         # which property to check
    sh:minCount 1 ;                   # must appear at least once
    sh:maxCount 1 ;                   # must appear at most once
    sh:datatype xsd:string ;          # value must be a string
    sh:message "A Title must have exactly one primaryTitle."
  ] ;

  sh:property [
    sh:path ont:isClassifiedAs ;
    sh:minCount 1 ;
    sh:maxCount 1 ;
    sh:class ont:ContentType ;        # value must be an ont:ContentType individual
    sh:message "A Title must be classified as exactly one ContentType."
  ] ;

  sh:property [
    sh:path ont:contains ;
    sh:class ont:Participation ;      # no minCount = optional, but type checked if
present
    sh:message "contains must point to a Participation."
  ] .
```

2.1 SHACL Constraint Parameters Used

Parameter	Meaning	Used for
sh:targetClass	Selects all individuals of this class as validation targets	All shapes
sh:path	The property being constrained	All property shapes
sh:minCount	Minimum number of values required (0 = optional)	Mandatory properties
sh:maxCount	Maximum number of values allowed	Single-valued properties
sh:datatype	Expected RDF datatype of the value	Data properties
sh:class	Expected class of the target individual (for object properties)	Object properties

sh:message	Custom error message shown when the constraint is violated	All property shapes
------------	--	---------------------

3. How the Shapes Were Derived from the Mapping

Each NodeShape was derived directly from the corresponding TriplesMap in the mapping file. The logic is straightforward:

- Every `rr:class` in a subjectMap becomes an `sh:targetClass` in a NodeShape.
- Every `rr:column` in a predicateObjectMap becomes a data property constraint with `sh:datatype xsd:string`.
- Every `rr:parentTriplesMap` in a predicateObjectMap becomes an object property constraint with `sh:class` pointing to the target class.
- Properties that come from mandatory SQL columns (NOT NULL) get `sh:minCount 1`. Properties from nullable columns get no `minCount` (optional).

R2RML construct	SHACL equivalent
<code>rr:class ont:Title</code> in subjectMap	<code>sh:targetClass ont:Title</code> in TitleShape
<code>rr:column "primary_title" (NOT NULL)</code>	<code>sh:path ont:primaryTitle ; sh:minCount 1 ; sh:datatype xsd:string</code>
<code>rr:column "job" (nullable)</code>	<code>sh:path ont:job ; sh:datatype xsd:string (no minCount)</code>
<code>rr:parentTriplesMap</code> <code><#MapContentType></code>	<code>sh:path ont:isClassifiedAs ; sh:class ont:ContentType</code>
<code>rr:parentTriplesMap</code> <code><#MapParticipation> (joinCondition)</code>	<code>sh:path ont:contains ; sh:class ont:Participation (no minCount — some titles may have no participations in sample data)</code>

4. Complete Shapes File — All 10 NodeShapes

4.1 Dictionary Class Shapes

These are the simplest shapes. Each dictionary class has exactly one mandatory name property with no optional properties. All are single-valued (maxCount 1).

```
shape:RoleShape a sh:NodeShape ;
  sh:targetClass ont:Role ;
  sh:property [
    sh:path ont:roleName ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A Role must have exactly one roleName."
  ] .

shape:GenreShape a sh:NodeShape ;
  sh:targetClass ont:Genre ;
  sh:property [
    sh:path ont:genreName ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A Genre must have exactly one genreName."
  ] .

shape:ContentTypeShape a sh:NodeShape ;
  sh:targetClass ont:ContentType ;
  sh:property [
    sh:path ont:content_type_name ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A ContentType must have exactly one content_type_name."
  ] .

shape:TitleTypeShape a sh:NodeShape ;
  sh:targetClass ont:TitleType ;
  sh:property [
    sh:path ont:title_type_name ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A TitleType must have exactly one title_type_name."
  ] .

shape:CategoryShape a sh:NodeShape ;
  sh:targetClass ont:Category ;
  sh:property [
    sh:path ont:categoryName ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A Category must have exactly one categoryName."
  ] .

shape:RegionShape a sh:NodeShape ;
  sh:targetClass ont:Region ;
  sh:property [
    sh:path ont:regionName ; sh:minCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A Region must have at least one regionName."
  ] .
```

4.2 Core Entity and Bridge Shapes

```
shape:TalentShape a sh:NodeShape ;
  sh:targetClass ont:Talent ;
  sh:property [
    sh:path ont:talentName ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "A Talent must have exactly one talentName."
  ] ;
  sh:property [
    sh:path ont:participatesIn ; sh:class ont:Participation ;
    sh:message "participatesIn must point to a Participation."
  ] .

shape:ParticipationShape a sh:NodeShape ;
  sh:targetClass ont:Participation ;
  sh:property [
    sh:path ont:as ; sh:minCount 1 ; sh:class ont:Category ;
    sh:message "A Participation must have at least one ont:as pointing to a Category."
  ] ;
  sh:property [
    sh:path ont:plays ; sh:class ont:Role ;
    sh:message "plays must point to a Role."
  ] ;
  sh:property [
    sh:path ont:job ; sh:datatype xsd:string ;
    sh:message "job must be a string."
  ] .

shape:AlternativeTitleShape a sh:NodeShape ;
  sh:targetClass ont:AlternativeTitle ;
  sh:property [
    sh:path ont:alternativeTitleName ; sh:minCount 1 ; sh:maxCount 1 ;
    sh:datatype xsd:string ;
    sh:message "An AlternativeTitle must have exactly one alternativeTitleName."
  ] ;
  sh:property [
    sh:path ont:isFrom ; sh:class ont:Region ;
    sh:message "isFrom must point to a Region."
  ] ;
  sh:property [
    sh:path ont:hasType ; sh:class ont:TitleType ;
    sh:message "hasType must point to a TitleType."
  ] .
```


5. Running the Validation

The validation was executed using the `shacl` command from Apache Jena, which is included in the Jena binary distribution (jena-6.0.0). The command uses forward slashes because the Windows command prompt does not accept backslashes in IRI-like paths:

```
shacl validate
--shapes "C:/path/to/shapes.ttl"
--data  "C:/path/to/out.ttl"
```

5.1 Validation Result

The output of the validation command was:

```
PREFIX rdf:
PREFIX sh:

[ rdf:type      sh:ValidationReport ;
  sh:conforms   true
] .
```

sh:conforms true means every individual in every class satisfies all constraints defined in `shapes.ttl`. Specifically this confirms:

- Every `ont:Title` individual has exactly one `ont:primaryTitle` string and exactly one `ont:isClassifiedAs` link to a `ContentType`.
- Every `ont:Talent` individual has exactly one `ont:talentName` string.
- Every `ont:Participation` individual has at least one `ont:as` link to a `Category`.
- Every dictionary individual (`Role`, `Genre`, `Category`, `ContentType`, `TitleType`, `Region`) has its `name` property populated with a string value.
- All object properties point to individuals of the correct declared class.

Note: A validation report with `sh:conforms false` would list each violation as a `sh:ValidationResult` node, showing the affected individual (`sh:focusNode`), the violated property (`sh:resultPath`), the constraint that failed (`sh:sourceConstraintComponent`), and the custom message (`sh:resultMessage`).

PART 2 — The SPARQL Demonstrator

The demonstrator is an interactive web application that allows live execution of SPARQL queries against the knowledge graph, without requiring Protege, a command-line tool, or any RDF-specific software from the user.

6. Why a Demonstrator?

A demonstrator serves a different purpose than SHACL validation or raw SPARQL execution. Its role is to make the knowledge graph **accessible and understandable** to an audience that may not be familiar with RDF or SPARQL syntax.

Concretely, it allows the project to be presented interactively: a query is selected, the result appears immediately in a readable table, and the audience can see the knowledge graph answer real questions about audiovisual data in real time.

6.1 Technology Choice

Several options were considered:

- **Protege SPARQL tab:** Already available, but extremely slow on datasets of this size and crashes frequently on complex queries with aggregation.
- **Apache Jena Fuseki:** A proper SPARQL endpoint server included in the Jena distribution. Requires starting a server, creating a dataset, and uploading the TTL file each session.
- **Custom Flask + rdflib app:** Chosen for this project. A Python Flask server loads out.ttl once at startup into an rdflib in-memory graph, and serves a minimal HTML interface. Fast, self-contained, no configuration needed between sessions.

Option	Pros	Cons
Protege SPARQL tab	No setup needed	Very slow, crashes on GROUP BY, not suitable for live demo
Apache Jena Fuseki	Full SPARQL 1.1, robust	Requires server setup each session, not beginner-friendly
Flask + rdflib	Fast startup, single command, full SPARQL 1.1 via rdflib	Requires Python + pip install

7. Demonstrator Architecture

7.1 Backend — app.py

The backend is a Python Flask application. At startup it loads out.ttl into an rdflib Graph object, which builds an in-memory triple store. It then exposes two HTTP routes:

- **GET /** — serves the index.html frontend file.
- **POST /query** — accepts a JSON body with a sparql key, executes the query against the rdflib graph, and returns the result as JSON with vars (column names) and rows (list of string values).

```
from flask import Flask, request, jsonify, send_from_directory
from rdflib import Graph

app = Flask(__name__)

g = Graph()
g.parse("out.ttl", format="turtle") # loaded once at startup

@app.route("/")
def index():
    return send_from_directory(".", "index.html")

@app.route("/query", methods=["POST"])
def query():
    sparql = request.json.get("sparql", "")
    try:
        results = g.query(sparql)
        vars_ = [str(v) for v in results.vars]
        rows = [[str(v) if v is not None else "" for v in row]
                 for row in results]
        return jsonify({"vars": vars_, "rows": rows})
    except Exception as e:
        return jsonify({"error": str(e)}), 400
```

7.2 Frontend — index.html

The frontend is a single self-contained HTML file with no external dependencies. It consists of three sections:

- **Query selector:** A dropdown listing all 13 pre-defined queries grouped by complexity (Simple, Non-Trivial). Selecting a query populates the editor textarea.
- **SPARQL editor:** A textarea where the query can be read, modified, or replaced with a custom query.
- **Results table:** A dynamically generated HTML table showing the query results returned by the backend. The row count is displayed above the table.

```
async function run() {
    const sparql = document.getElementById("editor").value.trim();
    const res = await fetch("/query", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ sparql })
    });
};
```

```
const data = await res.json();  
// render data.vars as headers, data.rows as table rows  
}
```

Note: The frontend communicates with the backend via a single `fetch()` POST call. No JavaScript libraries are required — the entire frontend is plain HTML, CSS, and vanilla JS.

8. SPARQL Query Design

The 13 queries loaded in the demonstrator were designed to cover every class and property in the ontology, and to demonstrate increasing levels of complexity. They are split into two groups in the interface.

8.1 Simple Queries (S1–S8)

Each simple query targets a single class and retrieves its individuals with their name property. They serve as sanity checks — non-empty results confirm that the corresponding TriplesMap executed correctly.

ID	Query	Class tested	Property tested
S1	Titles and start year	ont:Title	ont:primaryTitle, ont:startYear
S2	All genres	ont:Genre	ont:genreName
S3	All content types	ont:ContentType	ont:content_type_name
S4	All roles	ont:Role	ont:roleName
S5	All categories	ont:Category	ont:categoryName
S6	All regions	ont:Region	ont:regionName
S7	Titles with content type	ont:Title + ont:ContentType	ont:isClassifiedAs
S8	Alternative titles	ont:Title + ont:AlternativeTitle	ont:isAka

8.2 Non-Trivial Queries (NT1–NT5)

ID	Query	Why non-trivial
NT1	Title profile summary	Uses COUNT(DISTINCT), GROUP_CONCAT, and OPTIONAL simultaneously on three separate branches from the same Title node. The OPTIONAL variable is wrapped in COALESCE() to avoid an rdflib aggregation bug with unbound variables.
NT2	Versatile talents	Traverses Talent → Participation → Role and applies HAVING (COUNT(?roleName) > 2) to find multi-role individuals. Directly validates the talent_role SQL join in the mapping.
NT3	Multilingual AKA titles	Traverses Title → AlternativeTitle → Region with OPTIONAL extension to TitleType. Validates three junction mappings and confirms nullable region_id handling.
NT4	Roles x categories breakdown	Simultaneously traverses Participation → Role and Participation → Category, demonstrating that the two dimensions (from talent_role and title_principal respectively) are correctly separated in the graph.

NT5	Genre x content type distribution	Joins Genre ← Title → ContentType and uses GROUP_CONCAT to collapse content types per genre, showing the full breadth of the knowledge graph in a single analytical result.
-----	-----------------------------------	---

9. Known rdflib Limitation — OPTIONAL + GROUP_CONCAT

During development, query NT1 (Title profile summary) raised the following error when run through the Flask backend:

```
Error: Variable genreName is not bound
```

This error is a known limitation of rdflib's SPARQL engine. When a variable appears inside an OPTIONAL block and is also used inside a GROUP_CONCAT aggregation, rdflib raises an error because the variable may be unbound for some rows (those where the OPTIONAL pattern did not match).

The standard SPARQL 1.1 specification allows this usage — the variable should simply be treated as absent for rows where it is unbound. Apache Jena ARQ handles this correctly. rdflib does not.

9.1 The Fix — COALESCE

The fix is to wrap the optional variable in a COALESCE() call, which replaces any unbound value with a default — in this case an empty string. This guarantees the variable is always bound, removing the error:

```
# Before (fails in rdflib):
(GROUP_CONCAT(DISTINCT ?genreName ; separator=", ") AS ?genres)

# After (works in rdflib):
(GROUP_CONCAT(DISTINCT COALESCE(?genreName, "") ; separator=", ") AS ?genres)
```

Note: This workaround is only necessary for the Flask+rdflib demonstrator. The same query runs correctly without COALESCE in Apache Jena ARQ, which was used for all command-line SPARQL testing.

10. How to Run the Demonstrator

The demonstrator requires Python 3 and two packages: Flask and rdflib. All three files (app.py, index.html, out.ttl) must be in the same folder.

Step 1 — Install dependencies (one time only)

```
pip install flask rdflib
```

Step 2 — Start the server

```
python app.py
```

The terminal will print:

```
Loading out.ttl...
Loaded 12453 triples.
Open http://localhost:5000 in your browser
```

Step 3 — Open the interface

Open any browser and navigate to:

```
http://localhost:5000
```

Step 4 — Run a query

- Select a query from the dropdown (Simple or Non-Trivial group).
- The query is loaded into the editor textarea.
- Optionally edit the query.
- Click the Run Query button.
- Results appear in the table below within one or two seconds.

File	Role	Must be in same folder?
app.py	Flask backend — loads out.ttl, serves index.html, handles /query endpoint	Yes
index.html	Frontend — query selector, SPARQL editor, results table	Yes
out.ttl	The RDF knowledge graph generated by the R2RML mapper	Yes

End of document.